

Speaker Notes

Contents

- [1 1. Git and Source Code Version Control](#)
- [2 2. Three messages you have already seen](#)
- [3 3. Roadmap](#)
- [4 4. Takeaways](#)
- [5 5. What is here, and what is not](#)
- [6 6. What a commit actually is](#)
- [7 7. Git is a filesystem with a version-control interface on top](#)
- [8 8. Four object types, and nothing else](#)
- [9 9. Open `.git` and look](#)
- [10 10. A blob is content without a name](#)
- [11 11. The index is the third state](#)
- [12 12. A tree gives content its names](#)
- [13 13. A commit is a tree, its parents and a message](#)
- [14 14. A ref is a file with a hash in it](#)
- [15 15. Predict: what did those five commands build?](#)
- [16 16. Observe](#)
- [17 17. Explain: `add`, `commit` and `branch` were these all along](#)
- [18 18. Why `.git` is not enormous](#)
- [19 19. Snapshots are the model, packfiles are the storage](#)
- [20 20. The same two lines can hash to two values](#)
- [21 21. The hash detects accidents](#)
- [22 22. Add objects, or move refs](#)
- [23 23. Pointers, the graph, and one remote](#)
- [24 24. A branch is a pointer, not a container](#)
- [25 25. Ancestor, and the merge base](#)
- [26 26. A fast-forward creates nothing](#)
- [27 27. Detached HEAD is one file holding a hash](#)
- [28 28. A merge is a function of three trees](#)
- [29 29. A conflict is a line-range overlap](#)
- [30 30. The base can be plural, and then one side is discarded](#)
- [31 31. Three ways to call the same function](#)
- [32 32. Predict: which of the three leaves `claire` behind?](#)
- [33 33. Observe](#)

34 34. Explain: one function, three placements

35 35. Why a squash-merged branch keeps re-conflicting

36 36. Rebase keeps the tree and replaces the commit

37 37. Garbage collection deletes what no ref reaches

38 38. A remote is a name and a refspec

39 39. `origin/main` is a local file

40 40. "Up to date" is a claim about your last fetch

41 41. `fetch` moves remote-tracking refs and nothing else

42 42. `pull` refuses when the branches have diverged

43 43. `push` asks, and the remote applies one rule

44 44. Predict: a colleague pushed while you worked

45 45. Observe

46 46. Explain: the refusal is a report

47 47. Where centralisation went

48 48. What a central server used to provide

49 49. Distribution removed the fact, not the need

50 50. Locking needs a single arbiter

51 51. Per-path read access has no Git equivalent

52 52. Git does ship enforcement, and it binds one repository

53 53. Can Git plus forge policy meet the requirement?

54 54. What Git costs, and who avoids paying it

55 55. Nothing in Git's design was the only option

56 56. The index has two roles, and only one needs to be visible

57 57. Keep the object graph, replace the layer above it

58 58. `cherry-pick` writes a new object with no link to the original

59 59. Darcs and Pijul make the change the object

60 60. Git stores no fact that one file became another

61 61. A rename plus a large edit scores below the threshold

62 62. Scale breaks one assumption, not the model

63 63. Trace one surprise to the choice that produced it

64 64. Branching strategy

65 65. A branching strategy is a contract about branch lifetime

66 66. The five models you will meet

67 67. Three merge methods, three shapes of history

68 68. Trunk-based development is a cadence, not a topology

69 69. Two variants, split by team size

70 70. Two preconditions that summaries leave out

71 71. Conflicts rise with divergence, and size predicts better than time

72 72. The resolutions themselves ship defects

73 73. The delivery evidence points the same way, and is weaker

74 74. Nobody defends long-lived feature branches

75 75. Flags replace branches, and the debt is measured

76 76. Which branching strategy does your project earn?

77 77. Pull requests and review

78 78. The name is from 2005, the button is from 2011

79 79. What reviewers write about is mostly not defects

80 80. The outcome evidence cuts the other way

81 81. Review is comprehension, and comprehension is the bottleneck

82 82. Two numbers, one source, and they are not in conflict

83 83. A branch is old because it is waiting

84 84. Git cannot record that one branch depends on another

85 85. Two operations, under many names

86 86. Squash-merging one layer strands the layers above it

87 87. Review when nobody wrote the code

88 88. One pull request, or a stack?

89 89. Integration, delivery, deployment

90 90. Three practices wearing one abbreviation

91 91. Continuous integration is a frequency claim

92 92. Three questions that settle it

93 93. The same practice under two names

94 94. Continuous delivery is a property of the artefact

95 95. Continuous deployment is a policy

96 96. Feature flags follow from the release policy

97 97. Deployment frequency stands in for batch size

98 98. Diagnose what a team actually practises

99 99. Takeaways, restated

100 100. What follows from the object graph

101 101. What this course leaves open

102 102. Where to read next

103 103. References

Generated by the agentic vault — an autonomous research and authoring harness. Every factual claim was gated against a cited source registry before this document was produced.

v1 · 4e9f6b455 · built 2026-10-08

Slides: 103

11. Git and Source Code Version Control

On screen:

PB158

A content-addressed object graph, and a way to move pointers into it. Branching strategy, review and delivery practice are your team's policy.

Ondrej (Ondra) Krajicek me@ondrejkrjicek.com · www.linkedin.com/in/OndrejKrajicek

Built at render time

- Git is a content-addressable filesystem with a version-control interface on top [V1] [S2].
- Branching strategy, pull requests, review and delivery practice are policy layered on the data model [V10].

Title slide.

2 2. Three messages you have already seen

On screen:

Where this starts

Each one is an exact statement about a small data structure. None of them is an opinion.

1

Push refused

! [rejected] main -> main (non-fast-forward)

2

Pull refused

fatal: Need to specify how to reconcile divergent branches.

3

A warning after checkout

You are in 'detached HEAD' state.

Today these read as obstacles. By the end they read as reports about pointers.

- Teachers at one university report that learners do not think about the commands and just try them out [S20].
- The three messages above are documented behaviour, not defects [S6] [S12] [V4].
- Each message becomes predictable once objects and refs are on screen [inference].

Ask which one they met.

3 3. Roadmap

On screen:

Eight parts. The model first, team practice second, a decision at each end.

I · What a commit is

Four object types, by hand.

II · Pointers and the graph

Three ways to combine two branches.

III · Centralisation moves

Out of the data model.

IV · What Git costs

Rival designs.

V · Branching strategy

One variable.

VI · Review

What measurements show.

VII · Delivery practice

Three things, one name.

VIII · What stays open

What is not reached.

- Parts one to four build the model. Parts five to seven are the policy layer built on it [inference].
- Parts three to seven each end with a decision rule the room can apply to a real team [inference].

The payoff is stated early.

4 4. Takeaways

On screen:

1

Objects and pointers

A commit is a snapshot. A branch is a pointer.

2

One merge function

Three ways to combine two branches, three graphs.

3

Trade-offs, not laws

Every Git design choice had an alternative, and some system took it.

4

Branch lifetime

A contract about how long work stays unmerged.

5

What review produces

Understanding first, defects second, none found after 90 minutes.

6

Three practices, one name

Each one needs a different branching strategy.

- The six claims this topic exists to leave behind [T1] [T2] [T3] [T4] [T5] [T6].

Read all six aloud.

5 5. What is here, and what is not

On screen:

Scope

Covered

The object model, rebuilt with low-level commands.

Three ways to combine two branches, and one remote.

Branching strategy, review and delivery practice.

Two systems that made the opposite design choice.

Not covered

Any graphical client. The command line only.

Any hosting service's wider interface.

Interactive rebase corner cases, submodules, `git worktree`.

Multiple remotes and forks.

- Scope statement. The command line is the only interface used, per the course constraint [inference].
- Part VIII returns to the excluded items and names them as open [inference].

Scope stated before Part I.

6 6. What a commit actually is

On screen:

Part I

Four object types, one kind of pointer, and a repository rebuilt by hand.

- Part divider. Part I establishes the object model that every later part reads from [inference].

7 7. Git is a filesystem with a version-control interface on top

On screen:

Part I · The object model

Content addressing: store bytes, get back a name derived from those bytes.

Give Git a file. It writes the bytes into `.git/objects/` under a name that is the hash of a short header plus the content. Ask for that name later and the same bytes come back.

Identical content always produces the same name. Two files with the same bytes are stored once, in every repository that holds them.

- Git is a content-addressable filesystem with a version-control interface written on top [V1] [S2].
- An object is written under a name derived from a header plus its content [S1].
- Identical content yields one object, so deduplication is automatic rather than an optimisation [S1].

The interface came later.

8 8. Four object types, and nothing else

On screen:

Part I · The object model

TYPE	WHAT IT HOLDS
<code>blob</code>	File content. No filename and no path.
<code>tree</code>	Directory entries. A mode, a type, a name and a hash each.
<code>commit</code>	One tree hash, zero or more parent hashes, an author and a message.
<code>tag</code>	A target hash, a tagger and a message.

A commit hash covers its tree hash and its parent hashes. A tree hash covers the hashes below it. So one commit hash names the whole project state and all history behind it.

- The object universe is four types, and the list is complete [S1].
- A blob holds content and carries no name [S1].
- Because each hash covers the hashes below it, one commit hash names the whole history [S4].
- The same hash-tree structure appears in the container image layers the room has already met [inference].

Name the earlier topic.

9 9. Open `.git` and look

On screen:

Part I · The object model

A fresh repository is one line of text, a directory of small files, and an empty object store.

[code]

`HEAD` names the commit you have checked out. Here it does so indirectly, by naming a branch. `refs` is a directory of small files, and `objects` is empty, because nothing has been stored yet.

- The four entries that matter are `HEAD`, `index`, `objects` and `refs` [S2].
- `HEAD` holds a symbolic ref, normally the text `ref: refs/heads/main` [S3].
- The object store starts empty, so every object appearing later was written by a command run in the room [inference].

Run this live.

10 10. A blob is content without a name

On screen:

Part I · The object model

Two low-level commands write one object and read it back.

[code]

The first two characters of the hash are the directory. The remaining thirty-eight are the filename. Nowhere in that object is a filename recorded.

Low-level commands like these two are called plumbing. The everyday commands you already type are called porcelain.

- `hash-object -w` writes a blob and prints its name [S1].
- The object path splits the hash into a two-character directory and a thirty-eight-character filename [S1].
- A blob stores content only, so filenames must come from somewhere else [S1].
- The low-level commands are called plumbing, and the user-facing ones are called porcelain [S2].

Say plumbing and porcelain once.

11 11. The index is the third state

On screen:

Part I · The object model

Between the files on disk and a commit sits one file that maps every tracked path to a hash.

The working tree is the checked-out files you can edit. The index is `.git/index`, a list of paths with the hash of the content staged for each.

git add computes a blob and writes the path and hash into the index. Nothing is committed yet.

- `.git/index` is the third state, between working tree and commit [V7].
- The index maps every tracked path to the hash of its staged content [V7].
- `git add` stages an existing file, and it does not create the file [S20].

Say it plainly.

12 12. A tree gives content its names

On screen:

Part I · The object model

Write an entry into the index, then turn the index into a tree object.

[code]

--cacheinfo writes an index entry directly, from a mode, a hash and a path, with no file on disk. 100644 is the mode: Git records only normal file, executable file and symbolic link, and no owner, group or permission bits.

- `write-tree` turns the current index into a tree object [S1].
- A tree entry is a mode, a type, a filename and a hash [S1].
- The only valid blob modes are `100644` , `100755` and `120000` [S1].

Do not skip `update-index` .

13 13. A commit is a tree, its parents and a message

On screen:

Part I · The object model

One more command wraps the tree. The result is a snapshot with provenance.

[code]

A first commit has no parent line. Every later commit records the hash of the commit it was built on. That single link is what makes history a graph.

Your own commit hash will differ. The author name and the timestamp are hashed too.

- A commit holds one tree hash, zero or more parent hashes, identity and a message [S1].
- The parent link is recorded in the child, so history is built backwards [S1].
- A root commit has no parent line at all [inference].
- A commit hashes identity and timestamps, so two people committing one tree get two hashes [S1].

Compare the two hashes.

14 14. A ref is a file with a hash in it

On screen:

Part I · The object model

Objects are immutable. The only mutable thing in a repository is a pointer.

[code]

A branch is that file. Creating one writes forty-one bytes. Deleting one removes a file. Nothing is copied, at any repository size.

- A branch is a file under `.git/refs/heads/` containing one hash [V3] [S3].
- Refs are mutable and objects are immutable [S17].
- `HEAD` names a branch, so committing rewrites the branch file that `HEAD` points at [S3].

Show the file.

15 15. Predict: what did those five commands build?

On screen:

Part I · Demonstration

Write your answer down before anything else runs. Answer with equalities, not with hex.

1

Object count

How many files are under `.git/objects` now?

2

Porcelain equivalent

Which everyday command does each of the five replace?

3

Second commit

Commit again with nothing changed, using `git commit --allow-empty`. Which hashes change?

- Prediction is written before observation, because the method depends on committing to an answer first [S43].
- Asking for equalities rather than hash values keeps the prediction answerable [inference].

Collect three answers first.

16 16. Observe

On screen:

Part I · Demonstration

[code]

Three objects: one blob, one tree, one commit. The second commit carries the same tree hash and a new commit hash.

- Five plumbing commands produce exactly three objects and one ref [S1] [V63].
- An empty commit reuses the tree object and creates one commit object [S1].
- The tree hash is unchanged because the content is unchanged [V5].

Run it live.

17 17. Explain: `add`, `commit` and `branch` were these all along

On screen:

Part I · Demonstration

git add

hash-object -w plus an index entry.

git commit

write-tree, then commit-tree, then move the branch file.

git branch

update-ref. One small file, one hash.

The five commands added objects and moved one pointer. None of them edited an object in place. That is the whole vocabulary of the tool.

- `git commit` is write-tree plus commit-tree plus update-ref [V62].
- A Git repository is objects plus refs, and every command adds objects or moves refs [V2] [T1].
- No command edits an object in place, so recovery later is a question about reachability [V2].

Say the takeaway aloud.

18 18. Why `.git` is not enormous

On screen:

Part I · Storage

Every commit holds the whole project, and the cost tracks only what changed.

A file that did not change hashes to the same blob, so the new tree points at the object that already exists. A directory that did not change also hashes to the same tree. Git therefore shares everything below it instead of copying it.

This is the mechanism behind container image layers. Each party sees what looks like a private copy. Both parties point at one physical original.

- An unchanged file yields an unchanged blob hash, so the new tree points at the existing blob [V5] [S4].
- Storage cost tracks changed content rather than project size per commit [V5].
- The same structural sharing is copy-on-write, reached by content identity rather than page tables [V50].

Name the earlier topic again.

19 19. Snapshots are the model, packfiles are the storage

On screen:

Part I · Storage

Both halves are true. A delta is a recipe for rebuilding one object from another plus the difference.

The model

A commit names a whole tree. There is no delta anywhere in the object graph.

The storage layer

Git compresses each object on its own as it writes it. Git also packs many objects into one file from time to time. Only inside a packfile does Git store objects as deltas against one another.

- Snapshots are the model and deltas are an implementation detail of the storage layer [V6].
- Git packs loose objects periodically, and only packed objects are delta-compressed [S5].
- Students who learned Subversion carry the delta model across, and readers of the packfile format decide the snapshot story was a lie [V6].

Say both halves.

20 20. The same two lines can hash to two values

On screen:

Part I · Storage

Content addressing hashes bytes. Line endings are bytes.

A file saved with carriage-return and line-feed endings is not byte-identical to the same text saved with line-feed endings. The hashes differ.

Git normalises line endings to line feed in the index when a file is checked in. The default on Windows checks files out with carriage returns. So two laptops agree on the hash only when the file reaches the index with the same bytes.

- Line endings are normalised to line feed in the index every time a file is checked in [S13].
- The default for the `eol` attribute is carriage-return-line-feed on Windows [S13].
- A hash printed on a Windows laptop can therefore differ from the one on the slide [inference].

Say this before comparing.

21 21. The hash detects accidents

On screen:

Part I · Integrity

Tamper-evidence is structural. Adversary resistance is a property of the hash function.

Change one byte below a commit and the commit hash changes. Two people who agree on a hash agree on the whole history behind it. An attack finds a collision: two inputs, one hash.

Git's own breaking-changes document names four attacks on the SHA-1 default. They are the SHAppening in 2015, SHAttered in 2017, a birthday-near-collision in 2019, and Shambles in 2020.

The default moves to SHA-256 for new repositories at version 3.0. News reporting of a maintainer summit gives April 2027 for that release. The project's own document carries no date.

- Everything is checksummed before it is stored and is referred to by that checksum [S4].
- Git's own document names the SHAppening, SHattered, Birthday-Near-Collision and Shambles as the reason to move to SHA-256 [S15].
- One news report of a maintainer summit dates the release to April 2027, and the project page carries no date [S16].

Do not overstate this.

22 22. Add objects, or move refs

On screen:

Part I · The invariant

Two operations. Every command in the tool is one of them, or both.

Adds objects

add , commit , fetch , merge . Additive, and safe on shared work.

Moves refs

branch , reset , checkout , push . A ref that moves backwards unnames work.

commit --amend , rebase and cherry-pick do both. Cherry-pick copies one commit onto another branch. All three build new objects and move a pointer, and none of them modifies the commit you started from.

- Every command is either add objects or move refs, or both [V2].
- Amend, rebase and cherry-pick build replacement objects rather than editing existing ones [V2].
- Classifying a command this way predicts whether it is safe after publishing [inference].

Repeat the invariant here.

23 23. Pointers, the graph, and one remote

On screen:

Part II

Merge, rebase and squash are one function. A remote adds one more namespace of pointers.

- Part divider. Part II applies the Part I invariant to every operation the room runs daily [inference].

24 24. A branch is a pointer, not a container

On screen:

Part II · Refs

Nothing is inside a branch. A branch names one commit, and that commit names its parents.

Switching branches rewrites `HEAD` and replaces the files in the working tree. It does not move you into anything.

A branch owns no commits. A commit is reachable from a branch, or it is not. Git computes reachability by walking parent links.

- A branch is a file under `.git/refs/heads/` containing one hash [V3].
- Learners commonly regard branches as folders in the file system, and might try to move into one with `cd` [S20].
- Membership is reachability through parent links, not containment [inference].

Keep saying reachable.

25 25. Ancestor, and the merge base

On screen:

Part II · The graph

Two words carry every rule in this part. Both are defined by walking parent links.

WORD	DEFINITION
<i>ancestor</i>	<i>Commit A is an ancestor of B when A is reached by following B's parent links.</i>
<i>merge base</i>	<i>A common ancestor of two commits with no better common ancestor below it.</i>

A merge base is not "where the branches split". The ancestry ordering defines it. Git states plainly that a pair of commits can have more than one.

- The merge base is a best common ancestor under the ancestry ordering [S7].
- Git states that there can be more than one best common ancestor for two commits [S7].
- Teaching the base as the split point cannot predict the plural case [inference].

Ancestry arrives before merge.

26 26. A fast-forward creates nothing

On screen:

Part II · The graph

When one side is an ancestor of the other there is nothing to reconcile.

If your branch tip is an ancestor of the other branch tip, Git moves your branch file forward to that commit. Git writes no new object and computes no merge.

This is the one integration that adds nothing to history. It is available only when the branch never diverged.

- In a fast-forward `HEAD` is updated to point at the named commit without creating an extra merge commit [S8].
- Git's glossary defines a fast-forward as a merge with a descendant, which is why the word governs the push rule too [S6].
- A fast-forward is a pure ref move under the Part I invariant [V2].

Name it now.

27 27. Detached HEAD is one file holding a hash

On screen:

Part II · The graph

It is not a mode, a state machine or an error. It is the contents of `.git/HEAD`.

[code]

Commit now and Git writes a commit object, but no branch file moves. Switch away and nothing points at that commit any more.

The warning Git prints here is a description of that one file. It is not a diagnosis.

- Detached HEAD is simply `HEAD` containing a raw hash instead of a symbolic ref [V4].
- Committing in this state writes an object that no ref names [inference].
- The commit is abandoned rather than deleted, and the reflog still records the hash [V9].
- The warning names the contents of `.git/HEAD`, which the transcript prints on the next line [inference].

The third opening message.

28 28. A merge is a function of three trees

On screen:

Part II · Merging

A base, and two sides. Two versions are not enough to decide what happened.

With only two versions, "this line is in A and not in B" is ambiguous. It could be an addition on one side or a deletion on the other. The base resolves it.

Git compares each side against the base. Where one side changed a region, that side wins. Where both changed the same region, the function has no rule and emits a conflict.

- A three-way merge takes a base and two sides, and the third input is what resolves the ambiguity [S7] [S8].

- A conflict occurs if both sides have changes in a common segment of lines [V12].
- Overlap of changed line ranges, not meaning, is the entire conflict criterion [V12].

Draw three inputs once.

29 29. A conflict is a line-range overlap

On screen:

Part II · Merging

The criterion is textual. A merge that succeeds is not a merge that is correct.

Two people can edit different lines of the same function and produce a clean merge that does not compile. The comparison is over line ranges, and nothing in it depends on what the lines do.

The reverse also holds. Two people can edit the same line with the same intent, and the function still refuses, because the ranges overlap.

- A conflict occurs if both sides have changes in a common segment of lines [V12].
- Only the heads and the merge base are considered, and the individual commits between them are not [S8].
- A clean merge is therefore a statement about text, not about behaviour [inference].

Tests decide, not merges.

30 30. The base can be plural, and then one side is discarded

On screen:

Part II · Merging

Two people resolve the same conflict in opposite directions. Now two commits qualify as the base.

Git names this a criss-cross. Both candidates are best common ancestors, and neither is better. `git merge-base` prints one of them and does not say which, while `git merge-base --all` prints both.

Picking either candidate makes the merge succeed cleanly and silently drops the other person's decision. The default merge strategy avoids that: it merges the two candidates into one synthesised base, and uses that as the third input.

- Git states that under criss-cross merges there can be more than one best common ancestor [S7].
- Without `--all` it is unspecified which best base is output [S7].
- Using either candidate as the base will give an incorrectly clean merge [V13].
- When more than one common ancestor can be used, the default strategy merges the ancestors into one tree and merges against that tree [S8].

Read `merge-base --all` aloud.

31 31. Three ways to call the same function

On screen:

Part II · Merging

The function is fixed. What changes is which inputs it gets and where its result is written.

[code]

Both branches moved after `base`, so neither tip is an ancestor of the other. Nothing here can fast-forward.

```
git merge claire
```

One new commit with two parents.

```
git rebase main
```

Replay claire's commits onto main, one by one.

```
git merge --squash claire
```

One new commit with one parent.

- A merge commit is tied to both branches by recording both tips as its parents [S8].
- Rebasing replays the commits one by one, and each replay is similar to a cherry-pick [S9].
- `--squash` produces the merge result without recording the second parent [S8].

- None of these are different algorithms, and their differences are entirely in the object graph they leave behind [V11].

Three names for one function.

32 32. Predict: which of the three leaves `claire` behind?

On screen:

Part II · Demonstration

Three integrations of the same two branches. Write your answers down before anything runs.

1

Content

Do all three put the same file content on `main` ?

2

Parents

How many parents does the new commit on `main` have, in each case?

3

Ancestry

Afterwards, is `claire` an ancestor of `main` ?

- The prediction is written before the observation, and the questions ask for equalities rather than hash values [S43].
- Question three separates content from ancestry, which is the distinction the demonstration exists to teach [inference].

Take a show of hands.

33 33. Observe

On screen:

Part II · Demonstration

[code]

- The merge commit records both tips as parents, so the side branch is an ancestor afterwards [S8].
- The squash commit produces the same tree as the merge, because `--squash` produces the merge result [S8].
- The squash commit records one parent, so the side branch is not an ancestor and the merge base is still the root [V14].
- Rebasing left the same commit message under a new hash, because a commit hashes its parents as well as its tree [V15].

One fresh repository each time.

34 34. Explain: one function, three placements

On screen:

Part II · Demonstration

OPERATION	WHAT IT LEAVES IN THE GRAPH	PART I INVARIANT
Merge commit	Both tips become parents, so ancestry is preserved.	Adds objects
Rebase	One new commit per replayed commit, then the branch pointer moves.	Adds and moves
Squash	One new commit with one parent. The side branch is not an ancestor.	Adds objects

All three put the same content on `main`. Git's own merge and rebase manuals describe all three. They differ in what the graph then says about where that content came from.

- All three leave the same file content, and they differ in the object graph they leave behind [V11] [T2].
- Only the merge commit makes the side branch an ancestor of `main` [S8].
- Ancestry is what the next merge base is computed from [S7].

Say ancestry once more.

35 35. Why a squash-merged branch keeps re-conflicting

On screen:

Part II · Merging

The content arrived. The ancestry did not.

Merge-base discovery walks parent links. A squash commit has one parent, so the side branch's commits are not ancestors of anything on `main`.

Merge the same branch again and the base is computed from a graph in which that work never landed. Git re-offers changes that are already present, and they conflict with themselves.

A branch lands when its content reaches `main`. The fix is one line. Delete the branch once its content has landed. Start the next change from `main`.

- The squash commit has the same tree, but the side branch is not its ancestor and the merge base is still the root [V14].
- The vendor documents the same hazard, warning that later pull requests can include commits already squashed into the base branch [S36].
- Deleting the branch after landing removes the situation rather than repairing it [inference].

State the cure too.

36 36. Rebase keeps the tree and replaces the commit

On screen:

Part II · Rewriting

A commit hashes its parents as well as its tree. Change the parent and you have a different object.

Replaying a commit onto a new base can produce exactly the same file content. The commit hash is still different, because the parent hash is part of what was hashed.

That is why every hash changes after a rebase. Rebasing work that other people already fetched forces them to reconcile two histories of the same changes.

- A replayed commit can have the same tree and still be a different object, because a commit hashes its parents [V15].
- The rebase manual renames the replayed results in its own diagram, rather than reusing the original names [S9].
- Rebase adds objects and moves a ref, which is safe locally and hazardous after publishing [V2].

Recovery comes next.

37 37. Garbage collection deletes what no ref reaches

On screen:

Part II · Rewriting

A rewrite abandons objects. Garbage collection is the later pass that removes them.

[code]

The reflog records every position a ref has held in this repository, and `main@{1}` names the position before the last one. Pointing a new branch at an old position makes the abandoned commits reachable again, so collection leaves them alone.

- Rewritten commits are abandoned rather than deleted until garbage collection, and the reflog recovers a lost hash [V9].
- A ref that points at an abandoned commit makes it reachable again [inference].
- The reflog is local, so it does not recover work that was never in this repository [inference].

Teach recovery immediately.

38 38. A remote is a name and a refspec

On screen:

Part II · Remotes

Not a connection and not a session. Two lines of configuration that `clone` writes for you.

[code]

The left side of the refspec names refs on the other repository, and the right side names refs in yours. The leading `+` allows the destination to be replaced even when that loses history, which is safe here because nothing of yours lives there. `origin` is simply the name `clone` chooses.

A bare repository is the usual other end. It holds the object store and the refs with no checked-out files.

- The refspec `+refs/heads/*:refs/remotes/origin/*` fetches every branch on the remote into the remote-tracking hierarchy [S11].
- A bare repository does not have a locally checked-out copy of any of the files under revision control [S6].
- The refspec is a rename rule applied silently on every fetch [S11].

No network is needed.

39 39. `origin/main` is a local file

On screen:

Part II · Remotes

A third namespace of refs, stored on your disk, describing somebody else's repository.

[code]

This is a remote-tracking ref. It records where the remote's branch was at your last fetch. It is not the remote, and it is not your branch.

- The remote-tracking ref is stored locally and describes the remote [V16].
- Almost every practical confusion in team use is a confusion about that namespace [V16].
- Git's glossary defines it as a ref used to follow changes from another repository [S6].

Write three namespaces down.

40 40. "Up to date" is a claim about your last fetch

On screen:

Part II · Remotes

Nothing in `git status` reads the other repository.

[code]

The two commands disagree, and `status` follows the local file. It reported neither ahead nor behind while the other repository was one commit ahead.

- The count is not approximately true, and its accuracy is exactly the freshness of the last fetch [V18].
- `git ls-remote` is the live query, and `git status` compares two local refs [V18].
- The transcript above was produced against a local bare repository, so no network was involved [inference].

Read both hashes aloud.

41 41. `fetch` moves remote-tracking refs and nothing else

On screen:

Part II · Remotes

One command. One visible effect, and it is smaller than the name suggests.

[code]

Exactly one line changed. The uncommitted edit survived, the untracked file survived, `main` did not budge. The count is right now because `fetch` refreshed the local ref that `status` reads.

- `fetch` adds objects and moves the remote-tracking refs, and touches neither local branches nor the working tree [V17].
- The fetch manual scopes itself to refs and the objects needed to complete their histories [S11].
- The behind count became correct because its input was refreshed, not because a live query was made [V18].

One ref moved.

42 42. `pull` refuses when the branches have diverged

On screen:

Part II · Remotes

Not a silent merge. With nothing configured, current Git stops and asks you to choose.

[code]

`pull` is a fetch followed by an integration step, and you choose the step. When the branches have not diverged, all three configured answers produce the same fast-forward.

Run the two `git config` queries first. A laptop where either setting already holds a value does not refuse.

- The refusal and the three configured answers are what to teach, and `pull = fetch + merge` is not [V19].
- The manual names fast-forward-only as the default, and the reproduction at version 2.55.0 shows the unset default refusing instead [S12] [V19].
- When the branches have not diverged, every configured answer produces the same fast-forward [inference].
- A repository with `pull.rebase` or `pull.ff` already set integrates instead of refusing [S12].

The second opening message.

43 43. `push` asks, and the remote applies one rule

On screen:

Part II · Remotes

The destination must be an ancestor of the commit you are sending.

The remote checks whether its current branch tip is an ancestor of your tip. If it is, moving the pointer forward loses no history. The remote then accepts the update.

If it is not, somebody else's commits are in the way. Moving the pointer would leave those commits unnamed, so the remote refuses.

- Only fast-forward updates are allowed, and the destination must be an ancestor of the source commit [S10].
- A non-fast-forward update will lose history, which is the manual's own stated reason [S10].
- The rule is applied by the receiving repository, which is also where an update hook would run [S6] [S14].

The first opening message.

44 44. Predict: a colleague pushed while you worked

On screen:

Part II · Demonstration

One bare repository, two clones, one shared branch. Write down what each command changes.

[code]

1

After `git fetch`

Which of `main`, `origin/main` and the working tree changes?

2

After `git pull`

Does it complete, and what does it ask you?

3

After `git push`

Accepted or refused, and by which rule?

- The demonstration runs against a local bare repository standing in for a remote, so no network is needed [inference].
- The prediction is committed to paper before the commands run [S43].

A second person appears.

45 45. Observe

On screen:

Part II · Demonstration

[code]

Fetch changed one ref. Pull asked a question. Push refused, and the message names the rule.

- Fetch moved only the remote-tracking ref [V17].
- Pull refused because the branches had diverged and nothing was configured [V19].
- Push was refused because the remote tip was not an ancestor of the local tip [S10].

Predict before each command.

46 46. Explain: the refusal is a report

On screen:

Part II · Demonstration

The remedy is to integrate and send a descendant, not to reach for a flag.

1

Read it

Somebody else's commits are between your base and the remote tip.

2

Integrate

Fetch, look at what arrived, then merge or rebase onto it.

3

Push again

Your new tip now has the remote tip as an ancestor.

--force-with-lease sends the update only when the remote ref still holds the value your remote-tracking ref records. Fetching immediately before it weakens the check, because you then lease against a value you have not looked at.

- A rejected push is a report, not an error [V20].
- `--force-with-lease` requires the remote ref's current value to match the local remote-tracking ref [S10].
- Fetching immediately before a lease push weakens it, so the order is fetch, inspect, then lease [V21].

Fetch, inspect, lease.

47 47. Where centralisation went

On screen:

Part III

Distribution removed a fact from the data model. It did not remove the need for it.

- Part divider. Part III separates what the tool cannot do from what a team's policy does not do [inference].

48 48. What a central server used to provide

On screen:

Part III · Before

One repository, on one server, and committing is the same act as publishing.

In a centralised system such as Subversion there is one repository, on one server. Committing sends the change to that server, so committing and publishing are the same act. Working offline is not possible for anything that needs history.

The server saw every operation, so it could also enforce two things. One was an exclusive lock on a file. The other was a rule about which paths a user may read.

- Committing a change implicitly publishes it, and the distributed tools decouple the two [V22].

- A single authoritative server is a structural fact of the centralised data model [V22].
- Subversion offers access controls down to the level of individual files, and Git does not [V25].

Thirty seconds is enough.

49 49. Distribution removed the fact, not the need

On screen:

Part III · The relocation

Every clone is a complete repository. None of them is authoritative by construction.

Nothing in the object model marks one copy as the real one. Authority becomes a claim that people agree to honour, and the agreement is enforced somewhere outside the data model.

That somewhere is a forge: a hosting service that stores one repository and applies rules to it. Branch protection on a forge is doing the job the central server used to do structurally.

- Git relocated centralisation from the data model into policy, where forges now implement it [V22].
- Branch protection rules define whether collaborators can delete or force-push, and set requirements for any push [S44].
- The dominant practice is a hosted authoritative repository, which is a social convention on a peer-to-peer model [V10].

Define forge here.

50 50. Locking needs a single arbiter

On screen:

Part III · The residue

A lock is a claim over a resource, and a claim is only meaningful if one party decides it.

Two clones can each record that the lock is held, and nothing in a peer-to-peer model settles which record is right. A peer-to-peer data model therefore cannot provide locking. A team either relies on a social norm, or puts a single server back in the path.

The implication runs one way only. Locking requires centralisation. Centralisation does not require locking, which is why some centralised systems make locks optional.

- Locking requires centralisation, and centralisation does not require locking [V24].
- A lock needs a single arbiter to be meaningful, so distributed systems rely on social norms instead [V24].
- Merge-by-default assumes a useful three-way merge exists for the file type, and locking assumes it does not [inference].

The file type decides.

51 51. Per-path read access has no Git equivalent

On screen:

Part III · The residue

The unit of transfer is a reachable object graph, so there is no point at which a per-path rule applies.

A centralised server answers each command by consulting a table of paths and users. Git answers a fetch by sending the objects needed to complete a history.

To hide one directory you would have to rewrite the history being served. The workarounds therefore change the unit instead: split the tree into separate repositories, or enforce at the forge.

- Subversion offers access controls down to the level of individual files, and Mercurial and Git do not [V25].
- The workarounds change the unit of transfer rather than adding a rule [V25].
- Forge enforcement covers the shared copy and nothing else [V23].

Not an omission.

52 52. Git does ship enforcement, and it binds one repository

On screen:

Part III · The limit

"Git has no access control" is too strong. Git ships hooks, and a hook can refuse a push.

Git ships hooks. A receiving repository can run an update hook that refuses any non-fast-forward push, or that consults a table before accepting a ref. Those are core Git, not forge features.

What Git does not do is choose the policy. And a hook binds the repository it lives in. Anyone who clones from you can bypass it entirely, and two peers who exchange commits directly are bound by nothing.

- The update hook can enforce a fast-forward-only policy and can implement access control without filesystem ownership [S14].
- Policy above a peer-to-peer model can approximate a central server for the shared remote, and cannot bind two peers talking directly [V23].
- The defensible claim is that Git supplies the mechanism and sets no policy [inference] [T3].

The counterexample is stated first.

53 53. Can Git plus forge policy meet the requirement?

On screen:

Part III · Decision

One authoritative history

GitHub branch protection

Yes. Protect the branch, and require every landing to be a fast-forward.

Policy

Exclusive lock on a binary asset

—

No. A lock needs one arbiter that peers cannot provide.

Structural

Read access restricted per path

—

No. Split the repository, or use a centralised system.

Structural

- Branch protection sets requirements for pushes, including passing checks and a linear history [S44].
- Locking requires centralisation, so no forge rule recovers it [V24].
- A peer-to-peer model cannot supply the single arbiter a lock needs, and no source states this of Git directly [inference].
- Per-path access control exists in Subversion and not in Git or Mercurial [V25].
- Splitting the repository or moving to a centralised system is therefore the only remedy left [inference].

Ask for one requirement.

54 54. What Git costs, and who avoids paying it

On screen:

Part IV

Each limitation, paired with the system that made the opposite choice and what that choice cost.

- Part divider. Part IV supplies the comparative evidence that Git's design is choices rather than laws [inference].

55 55. Nothing in Git's design was the only option

On screen:

Part IV · The comparison

Content addressing already existed, and a rival project began two weeks after Git.

Monotone stored objects by content hash before Git did. Torvalds tried it for a day and gave up on it, because its performance was so bad.

Mercurial's announcement is dated 20 April 2005. It states two aims: to be as simple as possible, and to be as scalable as possible.

So content addressing was not Git's invention. Git's properties are choices, and other projects took several of the alternatives.

- Monotone had interesting ideas and performance so bad that its evaluator abandoned it in a day [S19].
- The Mercurial announcement is dated 20 April 2005 and states the aims of simplicity and scalability [S42].
- Git's rename design was already being defended on its mailing list on 14 April 2005, so the two projects are contemporaries [S18] [inference].
- Content addressing predates Git, and a contemporary project made different choices, so these properties are not laws [inference].

Name both systems once.

56 56. The index has two roles, and only one needs to be visible

On screen:

Part IV · The index

A cache of filesystem information, and a staging buffer the user operates.

The first role is uncontroversial. It is what makes `git status` fast enough to run constantly.

The second role is a design choice. Git exposes it, and exposes it under three different names across its own documentation: index, cache and staging area.

- The index is a cache of filesystem information and also a user-visible staging buffer [S22].
- Git exposes the index to the user, and Jujutsu names it as a concept it deliberately drops [S22].

- An analysis of Git's concepts finds that the staged and the working versions are not orthogonal [S21].
- The index is the third state between the working tree and a commit [V7].

Refer back to Part I.

57 57. Keep the object graph, replace the layer above it

On screen:

Part IV · A system that chose differently

Jujutsu stores its data in a Git repository and discards Git's user model.

The working copy is itself a commit, snapshotted automatically. Because of that there is no index, and no stash either. A stash is Git's side pocket for unfinished edits, and an ordinary commit already does that job.

Everything the system changes sits strictly above the object graph, and it still interoperates with the same repositories. That is what makes it evidence about Git rather than a rival product.

It is pre-1.0, at release v0.45.1 of September 2026, and the project calls itself experimental. This is not migration advice.

- Everything Jujutsu changes sits above the object graph, which separates what Git is from what Git merely does [V26].
- There is no index, because an automatically committed working copy makes an index-like concept unnecessary [S22].
- The same analysis reads stashing as a patch for flaws in Git's design rather than a requirement of version control [V31].
- As of release v0.45.1 it remains pre-1.0 [V27].

Interface replaced, format kept.

58 58. `cherry-pick` writes a new object with no link to the original

On screen:

*Part IV · A different model**Git derives changes by comparing snapshots. A change has no identity of its own.**Copying a commit to another branch builds a new commit object. Its content may be identical and its hash is different, because the parent differs. Nothing records that the two are the same change.**So Git cannot tell you that a fix is already present somewhere else, and a later merge can offer the same change again.*

- A replayed or copied commit has a different hash, because a commit hashes its parents as well as its tree [V15].
- Rebase replays commits one by one, and each replay is similar to a cherry-pick [S9].
- Nothing in the object model links a copy to its original [inference].

The pairing comes next.

59 59. Darcs and Pijul make the change the object

On screen:*Part IV · A different model**Store patches with durable identities, and derive the tree by applying a set of them.**Once a change is a first-class object you can ask whether two changes commute. If A then B reaches the same state as B then A, the two are independent.**A change keeps its identity wherever it is applied, so cherry-picking is not a re-creation. A later merge recognises the patch rather than re-offering it.**The model is a real trade. Darcs still records its own exponential merge problem as unfixed, and Pijul has no stable release. No independent benchmark found in this research compares Pijul's merge speed with Git's.*

- Darcs and Pijul store the change itself, give it a durable identity, and derive the tree state by applying a set of them [V28].
- A change carries the same identity wherever it is applied, so cherry-picking is not a re-creation [V29].

- Commutation is what lets a merge work by set difference over identities rather than by comparing endpoint trees [V28].
- The Darcs conflict representation can make merges exponential, and its own page records the answer as no, dated 2012-08-02 [V30] [S41].
- No independent benchmark of Pijul's merge performance against Git on a large repository was found for this topic [inference].

Define commute out loud.

60 60. Git stores no fact that one file became another

On screen:

Part IV · Unpaired

The rename is reconstructed at comparison time, from content similarity alone.

The design position was stated in 2005 and defended since. Git is about tracking data, and not about tracking what happened to create that data.

At comparison time Git scores each deleted path against each created path for content similarity. It merges the pair into a rename when the score passes a threshold. The default threshold is 50 percent.

- Git is about tracking data, not about tracking what happened to create that data [S18].
- Rename detection merges a deleted and a created filepair when the contents are similar enough [S46].
- The algorithm that measures the extent of the changes decides what counts as similar enough [S46].
- The default similarity score is 50 percent, and no object records the result [S46].

State the design position first.

61 61. A rename plus a large edit scores below the threshold

On screen:

*Part IV · Unpaired**Put both in one commit and the trail of the file ends there.**[code]**The similarity score falls below 50 percent. Git reports a delete and an add, so**--follow has nothing to cross.**The remedy is one commit per action. Commit the rename on its own. Commit the edit afterwards. No alternative system surveyed here removes the underlying cost.*

- Rename detection runs while two commits are compared, so a large edit in the same commit defeats it [S46].
- A commit holding only the rename keeps the similarity score high, so `--follow` crosses it [inference].
- No alternative surveyed for this topic pairs with this limitation, so it is named as unresolved [inference].

Show the two-commit remedy.

62 62. Scale breaks one assumption, not the model

On screen:*Part IV · Scale**A commit is a whole-tree snapshot whether the tree holds fifty files or five million.**What breaks is an assumption layered on top of the model. The assumption is that one client can do three things cheaply:*

- *hold the whole tree*
- *transfer it on clone*
- *walk it on every status check*

*The production fixes split into two families. One fetches less, so the repository is genuinely incomplete. The other presents a complete working tree over an incomplete one. It fetches each file on demand.**A partial clone therefore gives up offline operation, because a missing object has to be fetched.*

- What breaks at monorepo scale is an assumption layered on top of the model [V32].

- The symptoms do not share one axis, so a single fix does not exist [V32].
- A partial clone defers blobs to a promisor remote, so the client has to be online [inference].

The work moved.

63 63. Trace one surprise to the choice that produced it

On screen:

Part IV · Decision

1

Detached HEAD

HEAD may hold a raw hash, so a commit advances no branch. Jujutsu's working copy is itself a commit.

2

A stash you lost

The stash is a side pocket outside history. Jujutsu keeps no index and no stash.

3

A cherry-pick that conflicted again

A change carries no identity. Darcs and Pijul hash the change itself.

Name one Git behaviour that has cost you an hour. Then name the design choice that produces it, and the system that chose otherwise.

- Detached HEAD is HEAD holding a raw hash instead of a symbolic ref [V4].
- Jujutsu snapshots the working copy as a commit, so that design needs no index and no stash [S22] [V26].
- A copied commit gets a new hash, and nothing in the object model links it to the original [V15] [inference].
- Darcs and Pijul give the change a durable identity, so a patch survives being moved [V28] [V29] [T3].

Take three answers.

64 64. Branching strategy

On screen:

Part V

Five named models, one variable underneath them, and the evidence for setting it low.

- Part divider. Part V is the first part in which nothing new happens to the object graph [inference].

65 65. A branching strategy is a contract about branch lifetime

On screen:

Part V · The variable

Git enforces none of this. A branch can sit unmerged for two years and no command refuses.

In these repositories the single shared branch is called `main`. The literature calls that branch the trunk.

A strategy says which branches exist and who may move them. It says how long a change may stay outside the trunk, and what it must pass on the way back. The team imposes every one of those constraints from outside the tool.

The named models differ mainly in that one variable, and almost every documented cost and benefit follows from it.

- A branching strategy is a social contract expressed in refs [V33].
- Almost every documented cost and benefit follows from that one variable [V34].
- Trunk is the literature's word for the single shared branch, also known as `main` or `master` [S26].

Name trunk here.

66 66. The five models you will meet

On screen:

Part V · The models

MODEL	LONG-LIVED BRANCHES	BUILT FOR
Trunk-based	<code>main</code> only, plus short release branches	Frequent delivery, high throughput
GitHub Flow	<code>main</code> only	Web applications that deploy on merge
GitLab Flow	<code>main</code> plus one branch per environment or version	Explicit promotion gates, several versions
Release Flow	<code>main</code> plus one branch per release	Several supported versions at large scale
Gitflow	<code>main</code> and <code>develop</code> , plus release and hotfix	Scheduled, explicitly versioned releases

Read the middle column. The difference between the five is which branches are allowed to live a long time, and that is the whole disagreement.

- Trunk-based development resists any pressure to create other long-lived development branches [S26].
- Gitflow keeps two long-lived lines, `main` and `develop`, plus supporting release and hotfix branches [S29].
- Release Flow keeps `main` as the single integration branch and cuts a release branch per iteration [S35].
- Almost every documented cost and benefit follows from that one variable [V34].

Five names, one axis.

67 67. Three merge methods, three shapes of history

On screen:

Part V · Mechanics

MERGE METHOD	WHAT LANDS ON THE TRUNK	AVAILABLE WHEN
<i>Fast-forward</i>	<i>The branch commits, unchanged. No new object at all.</i>	<i>The branch never diverged</i>
<i>Merge commit</i>	<i>All branch commits, plus one commit with two parents.</i>	<i>Always</i>
<i>Squash</i>	<i>One new commit. Intermediate commits do not survive.</i>	<i>Always</i>

A forge offers these as three buttons, and all three are ordinary Git operations.

- A fast-forward updates `HEAD` without creating an extra merge commit [S8].
- Squash merging does not preserve intermediate commits as separate commits on the base branch [S36].
- Rebase merging always updates committer information and creates new commit hashes [S36].

Tie back to Part II.

68 68. Trunk-based development is a cadence, not a topology

On screen:

Part V · The model

The definition is stated in merges per day, not in the shape of the graph.

Each developer divides their work into small batches and merges into the trunk at least once a day, and potentially several times.

trunkbaseddevelopment.com defines it by what it forbids: collaborating in a single branch and resisting pressure to create other long-lived development branches. Short-lived branches for review and build checks are inside the model, not outside it.

- Each developer divides work into small batches and merges into trunk at least once a day [S28].

- The model forbids other long-lived development branches rather than all branches [S26].
- A pull-request branch that lives four hours is trunk-based, and one that lives four weeks is not [V34].

A number follows.

69 69. Two variants, split by team size

On screen:

Part V · The model

Direct to trunk

Commit straight to the trunk. Checks run after the commit lands. Up to 15 people.

Short-lived branches

One branch per change, reviewed and checked before landing. Sixteen or more developers.

trunkbaseddevelopment.com puts the boundary at sixteen developers. It says the cut-off was 100 people before lightweight branching and the pull request, and is 15 now. Below the boundary a branch and a review for every change buy little, because few people commit. Above it, unverified commits land on the trunk and break it for everyone else.

- With sixteen or more developers the team is more productive with short-lived branches verified before landing [S27].
- The site says the cut-off moved lower with lightweight branching and the pull request, from 100 people to 15 [S27].
- Both variants are the same model, differing in when the check runs [inference].

Ask the group size.

70 70. Two preconditions that summaries leave out

On screen:

Part V · Preconditions

2 days

the site's own risk threshold for becoming a long-lived branch

1 author

per branch, or two when pair programming

A branch that three people push to for a week is not a short-lived feature branch. It is a long-lived branch carrying a short-lived branch's name.

The trunk must also stay shippable, which is a testing property rather than a branching one. This is the healthy-branch rule: automated checks on each commit, so that the branch has no known defects.

- The branch should only last a couple of days, and longer than two days risks a long-lived feature branch [S27].
- The developer count should stay at one, or two if pair programming [S27].
- A healthy branch means automated checks on each commit to ensure there are no defects on the branch [S30] [T4].

Say the takeaway aloud.

71 71. Conflicts rise with divergence, and size predicts better than time

On screen:

Part V · Evidence

The largest mining study of merge conflicts, with the effect size that qualifies it.

One study covers 182,273 merge scenarios in 80 projects. The comparison is against the study's own average conflict rate. Branches integrated in under a day produced about 40% fewer conflicts. Branches older than 30 days produced about 161% more.

The same paper reports the effect size for branch duration as small, at minus 0.3 on its own scale. It finds the number of changed files, commits and authors to predict conflicts more strongly. So size is the stronger predictor, and duration is largely standing in for it.

- The study covers 182,273 merge scenarios across 80 projects, with 40% fewer conflicts under a day and 161% more beyond 30 days [S38].

- The same paper reports a small effect size of minus 0.3 for branching duration [S38].
- Size is the stronger predictor and duration is largely a proxy for size [V35].

Put both figures up.

72 72. The resolutions themselves ship defects

On screen:

Part V · Evidence

A conflict is not merely annoying. The code that comes out of one is measurably worse.

Across 143 open-source projects and 36,122 merge commits, 19.32% produced conflicts that a person had to resolve.

Lines involved in a conflict were about twice as likely to be buggy as lines that were not.

Lines involved in a conflict of entangled syntax-tree changes were about 26 times more likely to need a bug-fixing commit than lines from trivial conflicts.

- 19.32% of 36,122 merge commits produced conflicts requiring human intervention [S39].
- Conflicted lines were about twice as likely to be buggy [S39].
- Semantically entangled resolutions were about 26 times more likely to need a bug-fixing commit than trivial ones [S39].

Open-source repositories only.

73 73. The delivery evidence points the same way, and is weaker

On screen:

Part V · Evidence

Two lines of evidence with different weaknesses. One objection does not remove both.

The survey research associates higher delivery performance with teams that keep three or fewer active branches and merge to the trunk at least daily.

It is self-reported and correlational. Nothing in the data establishes that adopting the practice causes the performance rather than accompanying the test automation that makes it survivable.

- The association is with three or fewer active branches and merging to trunk at least daily [S28].
- The methodology relies on self-reported data, which introduces documented biases in accuracy and social desirability [V46].
- The conflict studies and the survey fail in different ways, which is why the pair is stronger than either [inference].

Caveat beside the number.

74 74. Nobody defends long-lived feature branches

On screen:

Part V · The real dispute

The argument that survives is about release branches, and whether your product needs them.

Vincent Driessen published the best-known model in 2010, and it is usually called Git-flow. He narrowed it himself in 2020. For a team that ships on demand he suggests a much simpler workflow. For software that is explicitly versioned, or that supports multiple versions in the wild, he says his model may still fit.

Microsoft's documented practice does both at once. Short-lived branches come off `main`. The team cuts a release branch for each three-week iteration. That branch never merges back, and fixes reach it by copying a commit forward from `main`.

- Nobody defends long-lived feature branches, and the dispute is entirely about long-lived release branches [V36].
- Driessen suggests a simpler workflow for continuous delivery, and says his model may still fit explicitly versioned software [S29].
- Release branches never merge back, and a fix goes to the trunk first so that it does not recur at the next deployment [S35].

Fix trunk, then forward.

75 75. Flags replace branches, and the debt is measured

On screen:

Part V · The substitution

Unfinished work has to be isolated somewhere. A branch isolates it in history, a flag isolates it at run time.

An unmerged branch is listed by `git branch` until somebody deals with it. A feature flag is a switch in the code that hides unfinished work from the user. A flag left in place after a release is listed nowhere.

Across 100 projects holding 7,593 flags, half were removed within fifteen months, and a quarter remained for more than eight years. Accumulation is the default path, not an occasional lapse.

- Every unmerged branch announces itself, and a flag that was supposed to be removed does not [V37].
- Half of the flags are removed within 15 months, and 25% remain for a very long time or forever [S40].
- Cheap integration is what makes small changes possible at all, and the setup cost is what moved [V49].

Part VII decides this.

76 76. Which branching strategy does your project earn?

On screen:

Part V · Decision

1

Can checks run in minutes?

If not, short-lived branches deliver instability rather than speed.

2

One author per branch?

A shared branch is a long-lived branch with a short name.

3

*Versions live in the field?**A customer on an old version needs a patch. That earns a release branch.*

4

*How big is the change?**Duration is largely a proxy for size. Ask the size too.**Answer these four about your own group project. Two answers of no mean short-lived branches will not work yet.*

- The healthy-branch precondition is automated checks on every commit [S30].
- One author per branch, or two when pair programming [S27].
- Supporting multiple versions in the wild is what earns a long-lived release branch [S29] [V36].
- Changed files, commits and authors predict conflicts more strongly than branch duration does [S38] [V35].

Run it on one project.

77 77. Pull requests and review

On screen:

*Part VI**Not Git. A convention roughly eighteen years old, and what the measurements actually say about it.*

- Part divider. Part VI is about what people do, and the only Git fact in it is a limitation [inference].

78 78. The name is from 2005, the button is from 2011

On screen:

Part VI · Origins

A pull request was originally a request that somebody else perform a pull.

A maintainer with work ready mailed a message naming a repository and a list of commits, and asked the recipient to fetch it. Git shipped a command to generate exactly that message in July 2005.

In February 2008 GitHub wrapped that message in a web page. Inline comments on the difference arrived in February 2011, three years later. The button that performs the merge arrived in April 2011, two months after that. Until then every pull request was merged by hand.

- A pull request was originally a request that someone perform a pull, which is why the name does not describe what it does today [V39].
- `git request-pull`, the command that generates exactly that message, was written in July 2005 [V60].
- GitHub's February 2008 pull request was essentially a graphical wrapper over that command [V61].
- Inline comments in the diff view arrived in February 2011, and the merge button in April 2011 [V51].
- Before the merge button, every pull request was merged by hand [V52].
- Git ships approval, reviewer and comment thread nowhere, because those are forge inventions [V38].
- The data model was fixed in 2005 and the policy layer accreted between 2008 and 2011 [inference].

Chronology is the thesis.

79 79. What reviewers write about is mostly not defects

On screen:

Part VI · Evidence

One study classified 570 comments from 200 review threads at a large software company.

Finding defects was the top-ranked motivation the participants gave. In the classified comments, defect was the fourth most frequent category of nine, at 14%. Code improvements was the most frequent, at 29%.

The study's own summary says review comments about defects are few, at roughly one eighth of the sample. It adds that they mostly address superficial concerns rather than design-level ones.

That does not make review idle. A separate study still estimates up to two and five extra defects after release where review coverage and participation are low.

- Finding defects was the top-ranked motivation the participants gave [S23].
- In the classified comments, defect was the fourth most frequent category of nine, at 14% [S23].
- Code improvements was the most frequent category, at 29% [S23].
- Review comments about defects are few, comprising one-eighth of the total in that sample [S23].
- Those defect comments mostly address superficial concerns rather than conceptual or design-level ones [S23].
- The study observed 17 developers, classified 570 comments and surveyed 873 programmers [S23].
- What the study finds review delivering instead is knowledge transfer, team awareness and shared code ownership [S23].
- Low review coverage and participation are estimated to produce up to two and five additional post-release defects [S24].

Outcomes come next.

80 80. The outcome evidence cuts the other way

On screen:

Part VI · Evidence

Counting what reviewers write is not the same as measuring what review prevents.

The two-and-five estimate came from measuring components rather than comments. It set how much review each component received against how many defects that component showed after release.

So the honest claim is narrower than either headline. Review spreads understanding of the change: afterwards a second person can alter that code without asking the author. It also catches some defects, and only up to a size limit.

- Low review coverage and participation are estimated to produce up to two and five additional post-release defects respectively [S24].
- Review is a bounded-capacity comprehension activity whose dominant output is shared understanding [V40] [T5].
- Its measured defect yield degrades sharply once a change exceeds what one person holds in one sitting [V40].

Say the takeaway aloud.

81 81. Review is comprehension, and comprehension is the bottleneck

On screen:

Part VI · Mechanism

A reviewer does not fail to find a defect through carelessness. Finding one requires understanding the change.

The same study found that code and change understanding is the key aspect of reviewing. It also found that 798 of 873 surveyed programmers confirmed that unfamiliar files take longer.

This is the mechanism underneath every rule about small changes. Small changes are not tidier. They are comprehensible.

- Code and change understanding is the key aspect of code reviewing [S23].
- 91% of the surveyed programmers confirmed that unfamiliar files take longer to review [S23].
- Every size rule in this part follows from the comprehension limit rather than from taste [inference].

Mechanism before numbers.

82 82. Two numbers, one source, and they are not in conflict

On screen:

Part VI · Limits

100–300

lines in 30 to 60 minutes, the authors' own advice

400

lines, the ceiling, because more overwhelms a reviewer

A ten-month study of 2,500 reviews covering 3.2 million lines produced both figures.

The first pairs a size with a time. The second is a size limit on its own.

Time carries a separate finding in the same study. Reviewers found defects at a roughly constant rate through the first 60 minutes, after which the rate levelled off. No reviewer found a defect after 90 minutes.

- The advice is to review between 100 and 300 lines at a time and spend 30 to 60 minutes on it [S25].
- Lines under review should be under 200 and not exceed 400, because larger changes overwhelm reviewers [S25].
- The study ran for ten months and covered 2,500 reviews of 3.2 million lines of code [S25].
- Defects appeared at relatively constant rates through the first 60 minutes, after which the rate levelled off [S25].
- In no case is a defect discovered after 90 minutes, which is a finding separate from the line ceiling [S25].

One source, two figures.

83 83. A branch is old because it is waiting

On screen:

Part VI · Queues

Review latency does not merely delay a change. It grows the change it then fails to review.

A change sits waiting for a reviewer. The author has more work that depends on it, so the commits go onto the same branch rather than a new one.

The change that eventually reaches a reviewer is therefore larger than the one that was submitted. Past 400 lines the review measurement found no further defects, so the waiting produces the size that review can no longer absorb.

- What developers usually do is add more commits to the waiting change, which turns it into a giant code change [V41].
- The size ceiling means the enlarged change is past the point where defects are found [S25].
- Branch duration is largely a proxy for size, which connects this directly to the Part V evidence [V35].

Parts V and VI meet.

84 84. Git cannot record that one branch depends on another

On screen:

Part VI · The Git limitation

A commit records its parents. Nothing records its children.

A stack is one change split into a chain of branches, each built on the branch below it. There is no object that says branch B is stacked on branch A. Nothing in the repository records that editing the bottom of a chain invalidates every layer above it.

Every tool that offers stacked review closes that gap the same way: by keeping each branch's intended parent outside the commit graph. Every such tool therefore has a state that can disagree with the repository.

- A commit records its parents and records no child [V8].
- No Git object says that one branch is stacked on another [V42].
- Each stacking tool keeps the dependency outside the graph, so its state can drift [V42].

The only Git claim here.

85 85. Two operations, under many names

On screen:

Part VI · Stacking

Retarget

Point each change's comparison base at the change below it, so its difference shows only its own layer.

Restack

After any edit to a lower layer, replay every layer above it and move the intermediate branch tips.

Interactive rebase has carried the second operation since Git 2.38, as `git rebase -i --update-refs`. On 30 July 2026 GitHub put stacked pull requests into public preview, which is a trial release rather than the finished feature.

- A stack is a chain of changes, each targeting the branch of the one below it [S37].
- The feature is rolling out in public preview to all repositories rather than generally available [S37].
- Git 2.38 taught `git rebase -i` to update branches whose tip appears in the rebased range, with `--update-refs` [S47].
- No source consulted here documents how a stack behaves inside a merge queue [inference].
- Landing part of a stack merges lower layers while the ones above stay open and rebase automatically [S37].

State the preview status.

86 86. Squash-merging one layer strands the layers above it

On screen:

*Part VI · Stacking**The Part II squash result, applied to a chain.**Squash the bottom layer and the trunk gains one commit with one parent. The original branch is not an ancestor of it.**The layer above still sits on the old version of the layer below. Its merge base falls back to the root, and it re-presents content that is already integrated.*

- The layer above a squash-merged layer finds its merge base back at the root and re-presents integrated content [V43].
- The squash commit carries the same tree, and the side branch is not its ancestor [V14].
- The vendor documents the same hazard for ordinary sequential pull requests [S36].

Ask them first.

87 87. Review when nobody wrote the code

On screen:

*Part VI · What changes next**The measured output of review assumes a person wrote the change and a person absorbs the comment.**If a language model produced the change, nobody on the team ever held the reasoning behind it, so there is no understanding to transfer. Catching defects is then the only function review has left, and the measured size and time limits bound that function.**The measurement to watch is where the effort goes. In one 2026 developer survey the work moved from writing code to verifying it, rather than disappearing. That is an open question, not a settled result.*

- The work moved from writing to verifying rather than disappearing [V48].
- The knowledge-transfer output assumes a human author and a human reader, and neither premise is guaranteed [inference].
- No source consulted here measures review effectiveness on generated code specifically [inference].

Leave it open.

88 88. One pull request, or a stack?

On screen:

Part VI · Decision

1

Under 300 lines

One change. A stack costs coordination it does not need.

2

Separable layers

Stack it, so each layer is reviewed against the one below.

3

Landing a stack

Merge bottom upwards. Squash an intermediate layer and the rest re-conflicts.

- The recommendation is 100 to 300 lines at a time, and the ceiling is 400 [S25].
- Landing part of a stack merges lower layers first, and the layers above rebase and retarget [S37].
- Squash-merging an intermediate layer strands the layers above it [V43].

Apply it to one change.

89 89. Integration, delivery, deployment

On screen:

Part VII

Three different practices, one abbreviation, and a branching strategy that depends on which one you do.

- Part divider. Part VII closes the loop, because the branching choice in Part V is bought by this practice [inference].

90 90. Three practices wearing one abbreviation

On screen:

Part VII · Separation

Continuous integration

A property of how a team uses version control.

Continuous delivery

A property of the artefact the pipeline produces.

Continuous deployment

A policy about what happens to an artefact that passes.

The pipeline is the automated sequence of builds and tests that every change goes through.

Each one requires the one before it. A team can integrate continuously and release twice a year. A team cannot deploy continuously without first being able to deliver continuously.

- Integration is a property of version-control use, delivery a property of the artefact, deployment a policy [V44].
- Continuous deployment requires continuous delivery first [S32].
- The other link in the chain is not stated by any source consulted here, and follows from the definitions [inference].
- The three are routinely collapsed into one abbreviation, which hides the dependency order [inference].

Leave three on the board.

91 91. Continuous integration is a frequency claim

On screen:

Part VII · Integration

It is about how often people merge, not about which server runs the build.

Each member merges their changes into the shared codebase together with their colleagues' changes at least daily. Each of those integrations is verified by an automated build that includes tests.

Two clauses, both routinely dropped. At least daily makes it a claim about cadence. Verified by an automated build including tests makes it a claim about a self-testing build.

- Each member merges into the codebase with colleagues at least daily [S31].
- Each integration is verified by an automated build including tests [S31].
- Running a build server does not establish the practice, because the cadence clause is separate [S31].

Say it plainly.

92 92. Three questions that settle it

On screen:

Part VII · Integration

1

Daily

Is everyone pushing to the trunk every day, rather than to feature branches?

2

Every commit

Does every commit trigger a run of the unit tests?

3

Ten minutes

When the build breaks, is it typically fixed within ten minutes?

The author of the test reports that in his experience fewer than 20% of teams who believe they are doing this can pass it. That figure is one practitioner's recollection, not a measurement.

- The three questions cover daily trunk pushes, unit tests on every commit and a ten-minute fix [S34].

- Less than 20% of teams that think they are doing CI can actually pass the test, in the author's stated experience [S34].
- Using a tool does not mean doing the practice [S34].

One anecdote, labelled.

93 93. The same practice under two names

On screen:

Part VII · The link back

The branching choice in Part V and the integration practice here are one decision.

The author of the canonical definition writes that this practice is called continuous integration, or in some circles trunk-based development.

That is why the preconditions match. Merging daily requires a trunk that stays ship-pable, which requires automated checks on every commit, which is the healthy-branch rule from Part V.

- The practice is called continuous integration, or in some circles trunk-based development [S31].
- A healthy branch means automated checks on each commit so that there are no defects on the branch [S30].
- Branching strategy therefore functions as a diagnostic for test maturity rather than an independent choice [inference].

Answer question one first.

94 94. Continuous delivery is a property of the artefact

On screen:

Part VII · Delivery

The team builds the software so that it can go to production at any time. Production is the running system users depend on, and nobody has to release to it.

The ThoughtWorks continuous delivery working group reduced it to four observable indicators.

- *The software is deployable throughout its life.*
- *The team prioritises keeping it deployable over new features.*
- *Anyone can get fast automated feedback on production readiness.*
- *Any version deploys to any environment on demand.*

The acceptance test is social. A sponsor could ask for the current development version to go to production at a moment's notice, and nobody would panic.

- Continuous delivery means the software can be released to production at any time [S32].
- The four indicators include prioritising deployability over new features [S32].
- The acceptance test is that a deployment request at a moment's notice would not cause alarm [S32].

Indicator two is expensive.

95 95. Continuous deployment is a policy

On screen:

Part VII · Deployment

Every change that passes the pipeline goes to production automatically.

Nothing about the artefact changes. What changes is who decides to release it, and the answer becomes nobody.

That is why it sits last in the dependency order. Releasing an artefact the team does not trust automatically reaches an outage faster than it reaches delivery performance.

- Continuous deployment means every change goes through the pipeline and is automatically put into production [S32].
- It requires continuous delivery first [S32].
- The distinction is the release decision, not the artefact [V44].

Ask about their project.

96 96. Feature flags follow from the release policy

On screen:

Part VII · The precondition question

Two sources disagree about whether flags are required. The disagreement dissolves along the delivery and deployment line.

The goal is to separate the technical decision to deploy from the business decision to launch a feature. Where releasing is automatic, the team must hide unfinished work at run time. That needs a feature flag, or a dark launch, which ships the code with nothing calling it yet.

Where a person still picks the release moment, that person is the gate, and the flag is optional. The author of the pattern says exactly this: flags are only really necessary when practising continuous deployment.

- Flags are needed when the release policy is automatic, and optional when a human still picks the release moment [V45].
- The goal is to separate the technical deployment decision from the business launch decision [S33].
- The author states that a feature flag is only really necessary when practising continuous deployment [S34].

Chosen on the release axis.

97 97. Deployment frequency stands in for batch size

On screen:

Part VII · The measurement

Batch size is how much change ships at once. It cannot be counted directly, so the number of releases stands in for it.

A smaller change fits inside the 400-line review limit from Part VI, and reverting it removes less than reverting a large one. Reducing the cost of a deployment is what makes smaller batches affordable at all.

The proxy is under strain. The 2024 survey report states that it is possible, even likely, that change sizes are growing, and it did not measure change size. Treat the proxy as a proxy.

- Reducing setup and deployment cost enables smaller release batches [V49].
- The report states that it is possible, even likely, that changelists are growing in size [V47].
- The report reasoned from a prior finding rather than measuring change size [V47] [T6].

Say the takeaway aloud.

98 98. Diagnose what a team actually practises

On screen:

Part VII · Decision

WHAT TO LOOK AT	THE QUESTION	WHICH PRACTICE
<i>Merge cadence</i>	<i>Daily to the trunk, or branches that live for weeks?</i>	<i>Integration</i>
<i>The artefact</i>	<i>Could any commit that passed the pipeline ship now?</i>	<i>Delivery</i>
<i>The trigger</i>	<i>Does passing the pipeline release, or does a person decide?</i>	<i>Deployment</i>

Fowler's definitions supply all three questions. A team that answers no to the first is not doing any of the three, whatever its pipeline is called.

- The cadence question is the definition of continuous integration [S31].
- The artefact question is the definition of continuous delivery [S32].
- The trigger question separates deployment from delivery [S32].
- A team failing the cadence question is doing none of the three, because each practice builds on the one before [inference].

Diagnose one public project.

99 99. Takeaways, restated

On screen:

1

Objects and pointers

A commit, rebuilt by hand. A branch is a file holding a hash.

2

One merge function

A squash kept the tree and lost the ancestry.

3

Trade-offs, not laws

Locking, per-path access, patch identity. Each has a rival design.

4

Branch lifetime

Two days, one author, checks on every commit.

5

What review produces

Understanding first, defects second, none found after 90 minutes.

6

Three practices, one name

Cadence, artefact, trigger. Ask all three.

- The same six claims, now paid for by the demonstrations and the evidence [T1] [T2] [T3] [T4] [T5] [T6].

Name the earning slide.

100 100. What follows from the object graph

On screen:

*Closing**The thesis, stated again now that every part of it has been shown.**Git gives you a content-addressed object graph and a way to move pointers into it. The object model has been stable since 2005.**Branching strategy, pull requests, code review and delivery practice are policy your team layered on top. They are choices, they are contested, and the evidence for them is weaker than the evidence for the graph. The honest limit: Git does ship hooks, so it supplies the enforcement mechanism and sets no policy.*

- A Git repository is objects plus refs, and every command adds objects or moves refs [V2].
- Pull requests, code review and continuous integration are not Git concepts [V10].
- Git ships hooks that can enforce a fast-forward-only policy, so the mechanism is Git and the policy is yours [S14] [inference].

Two sentences only.

101 101. What this course leaves open

On screen:*Closing*

•

*Forge-specific work**Part VI**The mechanism and the research, not one vendor's interface or its merge queue.*

•

*The wider remote surface**Part II**Multiple remotes, forks, custom refsspecs, and configuring the integration step.*

•

*History rewriting**Part II*

Interactive rebase corner cases, filtering a repository, submodules, work trees.

•

Spending the graph

Part II

git bisect searches the ancestry you now understand. It was not covered here.

- Each item is named rather than deferred, because no later topic in this course covers it [inference].
- The reflog was covered and bisect was not, and both read the same ancestry relation [inference].

Gaps named, not hidden.

102 102. Where to read next

On screen:

Closing

1

The object model

Pro Git, chapter 10. The plumbing sequence from Part I, in full.

2

The short version

The MIT Missing Semester lecture. One hour, and it states the model as type declarations.

3

The graph, visually

Learn Git Branching. A browser sandbox that draws the commit graph.

4

Why it is awkward

A conceptual-design analysis of Git's interface. Read it once the object model is in place.

Voluntary homework comes in three steps. Rebuild a commit by hand. Write all three predictions down. Then run the Part II demonstration.

- Chapter 10 of Pro Git carries the plumbing sequence used in Part I [S1] [S2].
- The Missing Semester lecture gives the data model as type declarations [S17].
- Learn Git Branching is a browser sandbox with paced exercises that draws the commit graph [S45].
- The conceptual-design paper is best read after the object model is in place [S21].

Set the voluntary homework.

103 103. References

On screen:

- [S1] Pro Git — Git Internals: Git Objects — Chacon & Straub. <https://git-scm.com/book/en/v2/Git-Internals-Git-Objects>
- [S2] Pro Git — Git Internals: Plumbing and Porcelain — Chacon & Straub. <https://git-scm.com/book/en/v2/Git-Internals-Plumbing-and-Porcelain>
- [S3] Pro Git — Git Internals: Git References — Chacon & Straub. <https://git-scm.com/book/en/v2/Git-Internals-Git-References>
- [S4] Pro Git — What is Git? — Chacon & Straub. <https://git-scm.com/book/en/v2/Getting-Started-What-is-Git%3F>
- [S5] Pro Git — Git Internals: Packfiles — Chacon & Straub. <https://git-scm.com/book/en/v2/Git-Internals-Packfiles>
- [S6] gitglossary(7) — The Git project. <https://git-scm.com/docs/gitglossary>
- [S7] git-merge-base(1) — The Git project. <https://git-scm.com/docs/git-merge-base>
- [S8] git-merge(1) — The Git project. <https://git-scm.com/docs/git-merge>
- [S9] git-rebase(1) — The Git project. <https://git-scm.com/docs/git-rebase>
- [S10] git-push(1) — The Git project. <https://git-scm.com/docs/git-push>
- [S11] git-fetch(1) — The Git project. <https://git-scm.com/docs/git-fetch>
- [S12] git-pull(1) — The Git project. <https://git-scm.com/docs/git-pull>
- [S13] gitattributes(5) — The Git project. <https://git-scm.com/docs/gitattributes>
- [S14] githooks(5) — The Git project. <https://git-scm.com/docs/githooks>
- [S15] BreakingChanges — The Git project. <https://git-scm.com/docs/BreakingChanges>

- **[S16]** *Git 3.0 gets a date* — Corbet, LWN (2026). <https://lwn.net/SubscriberLink/1094575/2385e98583715c2b/>
- **[S17]** *Version Control (Git)* — MIT Missing Semester. <https://missing.csail.mit.edu/2020/version-control/>
- **[S18]** *Re: Handling renames* — Torvalds, git mailing list (2005). <https://public-inbox.org/git/Pine.LNX.4.58.0504141102430.7211@ppc970.osdl.org/>
- **[S19]** *Google Tech Talk transcript* — Torvalds (2007). <https://archive.kernel.org/oldwiki/git.wiki.kernel.org/index.php/LinusTalk200705Transcript.html>
- **[S20]** *Making sense of distributed version control* — Isomottonen & Cochez. <https://www.cochez.nl/papers/dvcs-misconceptions-final.pdf>
- **[S21]** *What's Wrong with Git? A Conceptual Design Analysis* — Perez De Rosso & Jackson (2013). <https://groups.csail.mit.edu/sdg/pubs/2013/git-conceptual-onward13.pdf>
- **[S22]** *Jujutsu — Git comparison* — The Jujutsu project. <https://docs.jj-vcs.dev/latest/git-comparison/>
- **[S23]** *Expectations, Outcomes, and Challenges of Modern Code Review* — Bacchelli & Bird (2013). <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/ICSE202013-codereview.pdf>
- **[S24]** *The Impact of Code Review Coverage and Participation on Software Quality* — McIntosh, Kamei, Adams & Hassan (2014). https://rebels.cs.uwaterloo.ca/papers/msr2014_mcintosh.pdf
- **[S25]** *Best Kept Secrets of Peer Code Review* — Cohen, Teleki & Brown (2006). <https://static1.smartbear.co/support/media/resources/cc/book/best-kept-secrets-of-peer-code-review.pdf>
- **[S26]** *Trunk Based Development — Introduction* — trunkbaseddevelopment.com. <https://trunkbaseddevelopment.com/>
- **[S27]** *Short-Lived Feature Branches* — trunkbaseddevelopment.com. <https://trunkbaseddevelopment.com/short-lived-feature-branches/>
- **[S28]** *Capabilities: Trunk-based development* — DORA. <https://dora.dev/capabilities/trunk-based-development/>
- **[S29]** *A successful Git branching model* — Driessen (2010, revised 2020). <https://nvie.com/posts/a-successful-git-branching-model/>
- **[S30]** *Patterns for Managing Source Code Branches* — Fowler (2020). <https://martinfowler.com/articles/branching-patterns.html>
- **[S31]** *Continuous Integration* — Fowler (2023). <https://martinfowler.com/articles/continuousIntegration.html>

- **[S32]** Continuous Delivery — Fowler. <https://martinfowler.com/bliki/Continuous-Delivery.html>
- **[S33]** Patterns — Continuous Delivery — Humble. <https://continuousdelivery.com/implementing/patterns/>
- **[S34]** Continuous Integration — foundations — Humble. <https://continuousdelivery.com/foundations/continuous-integration/>
- **[S35]** How Microsoft develops with DevOps — Microsoft. <https://learn.microsoft.com/en-us/devops/develop/how-microsoft-develops-devops>
- **[S36]** About pull request merges — GitHub Docs. <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/incorporating-changes-from-a-pull-request/about-pull-request-merges>
- **[S37]** Stacked pull requests are now in public preview — GitHub Changelog (2026). <https://github.blog/changelog/2026-07-30-stacked-pull-requests-are-now-in-public-preview/>
- **[S38]** Attributes that may raise the occurrence of merge conflicts — Menezes et al. (2021). <https://doi.org/10.5753/jserd.2021.1911>
- **[S39]** An empirical investigation into merge conflicts and their effect on software quality — Brindescu et al. (2020). https://stairs.ics.uci.edu/papers/2020/emperical_MC.pdf
- **[S40]** Capture the Feature Flag: Detecting Feature Flags in Open-Source — Meinicke et al. (2020). <https://cmustrudel.github.io/papers/msr20flags.pdf>
- **[S41]** Darcs — Frequently Asked Questions (Performance) — The Darcs project. <https://darcs.net/FAQ/Performance>
- **[S42]** Mercurial v0.1 — a minimal scalable distributed SCM — Mackall (2005). <https://lkml.iu.edu/hypermail/linux/kernel/0504.2/0670.html>
- **[S43]** Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>
- **[S44]** About protected branches — GitHub Docs. <https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches>
- **[S45]** pcottle/learnGitBranching — Peter Cottle. <https://github.com/pcottle/learnGitBranching>
- **[S46]** gitdiffcore — Tweaking diff output — Git documentation. <https://git-scm.com/docs/gitdiffcore>
- **[S47]** Git 2.38 Release Notes — Git project (2022). <https://github.com/git/git/blob/v2.38.0/Documentation/RelNotes/2.38.0.txt>
- **[V1]** Git — Origin, Object Model, and Design Trade-offs — vault note.

- **[V2]** *Git — Origin, Object Model, and Design Trade-offs — vault note.*
- **[V3]** *Git — Origin, Object Model, and Design Trade-offs — vault note.*
- **[V4]** *Git — Origin, Object Model, and Design Trade-offs — vault note.*
- **[V5]** *Git — Origin, Object Model, and Design Trade-offs — vault note.*
- **[V6]** *Git — Origin, Object Model, and Design Trade-offs — vault note.*
- **[V7]** *Git — Origin, Object Model, and Design Trade-offs — vault note.*
- **[V8]** *Git — Origin, Object Model, and Design Trade-offs — vault note.*
- **[V9]** *Git — Origin, Object Model, and Design Trade-offs — vault note.*
- **[V10]** *Git — Origin, Object Model, and Design Trade-offs — vault note.*
- **[V11]** *Merge Algorithms — Three-Way Merge, Rebase, and First-Class Conflicts — vault note.*
- **[V12]** *Merge Algorithms — Three-Way Merge, Rebase, and First-Class Conflicts — vault note.*
- **[V13]** *Merge Algorithms — Three-Way Merge, Rebase, and First-Class Conflicts — vault note.*
- **[V14]** *Merge Algorithms — Three-Way Merge, Rebase, and First-Class Conflicts — vault note.*
- **[V15]** *Merge Algorithms — Three-Way Merge, Rebase, and First-Class Conflicts — vault note.*
- **[V16]** *Remotes, Remote-Tracking Refs, and Refspecs — What Fetch, Push, and Pull Actually Do — vault note.*
- **[V17]** *Remotes, Remote-Tracking Refs, and Refspecs — What Fetch, Push, and Pull Actually Do — vault note.*
- **[V18]** *Remotes, Remote-Tracking Refs, and Refspecs — What Fetch, Push, and Pull Actually Do — vault note.*
- **[V19]** *Remotes, Remote-Tracking Refs, and Refspecs — What Fetch, Push, and Pull Actually Do — vault note.*
- **[V20]** *Remotes, Remote-Tracking Refs, and Refspecs — What Fetch, Push, and Pull Actually Do — vault note.*
- **[V21]** *Remotes, Remote-Tracking Refs, and Refspecs — What Fetch, Push, and Pull Actually Do — vault note.*
- **[V22]** *Distribution moves centralisation from the data model into policy — vault note.*
- **[V23]** *Distribution moves centralisation from the data model into policy — vault note.*

- **[V24]** *Distribution moves centralisation from the data model into policy — vault note.*
- **[V25]** *Distribution moves centralisation from the data model into policy — vault note.*
- **[V26]** *Jujutsu — A Redesign of the Git User Model — vault note.*
- **[V27]** *Jujutsu — A Redesign of the Git User Model — vault note.*
- **[V28]** *Patch Theory — Darcs, Pijul, and Commuting Changes — vault note.*
- **[V29]** *Patch Theory — Darcs, Pijul, and Commuting Changes — vault note.*
- **[V30]** *Patch Theory — Darcs, Pijul, and Commuting Changes — vault note.*
- **[V31]** *Git's Conceptual Design Misfits — the Gitless Critique — vault note.*
- **[V32]** *Every Git Scaling Fix Either Fetches Less or Fakes the Rest — vault note.*
- **[V33]** *Branching Strategies — Trunk-Based vs Branch-Based Development — vault note.*
- **[V34]** *Branching Strategies — Trunk-Based vs Branch-Based Development — vault note.*
- **[V35]** *Branching Strategies — Trunk-Based vs Branch-Based Development — vault note.*
- **[V36]** *Branching Strategies — Trunk-Based vs Branch-Based Development — vault note.*
- **[V37]** *Branching Strategies — Trunk-Based vs Branch-Based Development — vault note.*
- **[V38]** *Code Review and the Pull Request as a Review Mechanism — vault note.*
- **[V39]** *Code Review and the Pull Request as a Review Mechanism — vault note.*
- **[V40]** *Code Review and the Pull Request as a Review Mechanism — vault note.*
- **[V41]** *Code Review and the Pull Request as a Review Mechanism — vault note.*
- **[V42]** *Stacked diffs — reviewing a change as a series of small dependent changes — vault note.*
- **[V43]** *Stacked diffs — reviewing a change as a series of small dependent changes — vault note.*
- **[V44]** *Continuous Integration and Continuous Delivery — vault note.*
- **[V45]** *Continuous Integration and Continuous Delivery — vault note.*
- **[V46]** *DORA Metrics — vault note.*
- **[V47]** *Deployment Frequency — vault note.*
- **[V48]** *DORA Metrics Limitations in AI-Assisted Development — vault note.*
- **[V49]** *The Small Batches Principle — vault note.*

- **[V50]** *Copy-on-Write — vault note.*
- **[V51]** *Code Review and the Pull Request as a Review Mechanism — vault note.*
- **[V52]** *Code Review and the Pull Request as a Review Mechanism — vault note.*
- **[V60]** *Code Review and the Pull Request as a Review Mechanism — vault note.*
- **[V61]** *Code Review and the Pull Request as a Review Mechanism — vault note.*
- **[V62]** *Git — Origin, Object Model, and Design Trade-offs — vault note.*
- **[V63]** *Git — Origin, Object Model, and Design Trade-offs — vault note.*

The references document, as a PDF and a web page, carries the full source list and a summary of every vault note with its own external references.

- Every source this topic relies on, in registry order [inference].

Summaries live there.